

Schritt 1: Technisches Setup in RStudio

Für unsere Analyse installieren wir das Paket `readxl`. Dieses Paket ermöglicht das Einlesen von Excel-Dateien (.xls und .xlsx) in R.

- Paket laden: Der erste technische Schritt im KDD-Prozess (Knowledge Discovery in Databases) ist der Import der Daten. Mit der folgenden Funktion laden wir die speziell für diese Aufgabe bereitgestellten Transaktionsdaten. Wir installieren zunächst einmal `install.packages("readxl")` über das Konsol.
- Laden der Bibliothek: Die Funktion `library()` lädt ein bereits installiertes Paket in die aktuelle R-Sitzung. Erst danach stehen dessen Funktionen – beispielsweise `read_excel()` – zur Verfügung.

Im Gegensatz dazu ist `split()` eine Funktion, die bereits fest in die Programmiersprache R integriert ist und kein extra Laden eines Pakets erfordert (Conversation History).

Um sicherzustellen, dass die in unserem Projekt verwendeten Pakete und deren exakte Versionen gespeichert und für die Zukunft festgehalten werden, installieren wir das Paket `renv` über die Console. Nach der Installation führen wir das Befehl `renv::init()` aus, um es in unserem Projekt integrieren.

Schritt 2: Datenimport

Im ersten Schritt des KDD-Prozesses werden die bereitgestellten Transaktionsdaten importiert.

- Die Funktion `read_excel()` liest die Excel-Datei ein und durch Zuweisung `meine_daten <-` werden die Daten dauerhaft in der Variable `meine_daten` gespeichert und können in den nächsten Analyseschritten weiterverwendet werden.
- Kommentare: Da unsere Datei in einem Unterordner namens „data“ liegt, nutzen wir einen relativen Pfad. Dabei schreiben wir den Ordernamen, gefolgt von einem Vorwärts-Schrägstrich (/) und dem Dateinamen in Anführungszeichen. `read_excel("data/Transaktionen_Apriori.xlsx")`

Wichtiger Hinweis: Wenn wir die Funktion `read_excel("data/Transaktionen_Apriori.xlsx")` ohne Zuweisung ausführen, wird das Ergebnis nur in der Konsole angezeigt und nicht für spätere Schritte (wie die Gruppierung) gespeichert (Conversation History). Das andere Problem ist, dass die Daten nicht dauerhaft gespeichert werden. Sobald die Ausgabe über den Bildschirm gelaufen ist, sind sie für R wieder „vergessen“. Deswegen führen wir mit Zuweisung `<-`, damit die Daten in unserem Environment-Fenster (oben rechts in RStudio) angezeigt zu bekommen.

```
meine_daten <- read_excel("data/Transaktionen_Apriori.xlsx")
```

Schritt 3: Vorbereitung der Daten (Transformation)

Da der Apriori-Algorithmus keine gewöhnliche Tabelle, sondern Transaktionen (Warenkörbe) erwartet, müssen die Daten zunächst transformiert werden.

- 3.1: Gruppierung der Daten (Transformation) Die Ausgangsdaten enthalten für jedes gekaufte Produkt eine eigene Zeile. Für die Warenkorbanalyse müssen jedoch alle Produkte, die zu derselben Bestellnummer gehören, zu einem gemeinsamen Warenkorb zusammengefasst werden. Hierfür wird die integrierte R-Funktion `split()` verwendet. Sie gruppiert die Produktgruppen anhand der Bestellnummer und erstellt daraus eine Liste von Warenkörben.

Erläuterung: produkte enthält alle Produktgruppen. bestellungen enthält die zugehörigen Bestellnummern. split() gruppiert sämtliche Produkte anhand ihrer Bestellnummer.

- Wir speichern die Bestandteile vorher in Variablen und dann gruppieren Produkte nach Bestellungen. Hier `trans_liste` ist eine Liste, aber noch kein Transaktionsobjekt.

```
produkte <- meine_daten$ProduktGroupName
bestellungen <- meine_daten$Bestellungsnummer

trans_liste <- split(produkte, bestellungen)
```

- 3.2 : Umwandlung in ein Transaktionsobjekt: Im nächsten Schritt wird die erzeugte Liste in ein Transaktionsobjekt umgewandelt. Hierzu wird das Paket `arules` verwendet. Dieses stellt die Daten in einem für den Apriori-Algorithmus geeigneten Format bereit, sodass die Transaktionen effizient verarbeitet und Assoziationsregeln ermittelt werden können.
- 3.2.2: wir installieren einmal `install.packages("arules")` über das Konsol
- 3.2.3: wir laden das Paket `library(arules)` in RStudio.

```
library(arules)
```

```
## Lade nötiges Paket: Matrix
```

```
##
```

```
## Attache Paket: 'arules'
```

```
## Die folgenden Objekte sind maskiert von 'package:base':
```

```
##
```

```
##      abbreviate, write
```

- 3.3: Transaktionsobjekt erzeugen: Das Paket `arules` erzeugt ein Transaktionsobjekt (transactions), das intern als spärliche Inzidenzmatrix (sparse incidence matrix) gespeichert wird. Dieses Format ermöglicht eine effiziente Verarbeitung durch den Apriori-Algorithmus.

```
warenkorb_trans <- as(trans_liste, "transactions")
```

- Kurze Überprüfung:

```
summary(warenkorb_trans)
```

```
## transactions as itemMatrix in sparse format with
```

```
## 10207 rows (elements/itemsets/transactions) and
```

```
## 38 columns (items) and a density of 0.08278374
```

```
##
```

```
## most frequent items:
```

```
##      Schutzblech      Mountainbike      Fahrradtasche      Schuhe E-Mountainbike
```

```
##           3805           2756           2375           2018           1940
```

```
##      (Other)
```

```
##           19215
```

```
##
```

```
## element (itemset/transaction) length distribution:
```

```
## sizes
##      1      2      3      4      5
## 800 2581 2668 2647 1511
##
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 1.000   2.000   3.000   3.146   4.000   5.000
##
## includes extended item information - examples:
##      labels
## 1 Accessoires
## 2          BMX
## 3          Brille
##
## includes extended transaction information - examples:
## transactionID
## 1           41112
## 2           41113
## 3           41114
```

Schritte 4: Data Mining (Warenkorbanalyse)

Nachdem die Daten erfolgreich in das Transaktionsformat überführt wurden, wenden wir nun den Apriori-Algorithmus an. Das Ziel ist es, logische Abhängigkeiten zwischen Produktgruppen zu erkennen, die häufig gemeinsam gekauft werden (Bundle-Angebote).

Definition der Parameter: Um die Analyse auf die relevantesten Muster zu fokussieren, haben wir folgende

Mindestsupport (0.05): Eine Produktkombination muss in mindestens 5 % aller Warenkörbe vorkommen, um berücksichtigt zu werden . Mindestkonfidenz (0.3): Die Regel muss eine Wahrscheinlichkeit von mindestens 30 % aufweisen (z. B.: „Wenn Produkt A gekauft wird, wird mit 30-prozentiger Wahrscheinlichkeit auch Produkt B gekauft“) . # 1. Den Apriori-Algorithmus ausführen # Wir generieren Regeln basierend auf Support und Konfidenz rules <- apriori(warenkorb_trans, parameter = list(supp = 0.05, conf = 0.3, target = “rules”))

2. Die Regeln nach ‘Lift’ sortieren

Der Lift-Wert zeigt an, wie viel stärker der Zusammenhang zwischen Produkten ist,

als es der reine Zufall erwarten ließe. Ein Lift > 1 ist ein starkes Indiz für eine Regel.

```
rules_sorted <- sort(rules, by = “lift”)
```

3. Die Top 10 Regeln anzeigen

Diese Regeln prüfen wir auf Validität, Neuartigkeit und Nützlichkeit.

```
inspect(head(rules_sorted, 10))
```